Sessió 8

Desenvolupament Web Full Stack (React, Node, Mongo)
Curs 2025-2026

Pràctica: E-commerce. Sessió 8

1. Dates....3
2. Objectius....3
3. Feina prèvia abans de la sessió....3
Avantatges de l'estratègia avançada d'autenticació amb JWT (Access + Refresh + Rotació)....3
1. Seguretat millorada....3
2. Sessions sense estat (stateless)....3
3. Rotació segura del refresh token....4
4. Protecció davant d'atacs de reutilització (replay attacks)....4
5. Control i revocació....4
6. Millor experiència d'usuari....4
4. Feina durant la sessió i abans de la següent sessió....5
4.1 Autenticaciò amb JWT + Token Refresh y Rotació Segura....5
Objectius:....5
- Implementar login amb generació de access token i refresh token....5
- Mantenir la sessió activa amb rotació de refresh tokens....5
- Protegir rutes amb middleware que validi els tokens....5
- Millorar la seguretat per a evitar atacs de reutilització (token theft)....5
Com implementar un sistema de login amb tokens JWT, mantenir la sessió activa amb rotació segura i protegir les rutes....5
1. Generar un access token i un refresh token en iniciar sessió....5
2. Mantenir la sessió activa amb rotació segura de tokens de refresc....5
3. Protegir les rutes amb middleware que valida els tokens....6
4. Millorar la seguretat contra atacs de reutilització (robo de tokens)....6
4.3 Implementar....7
4.4 Crear l'esquema....7
4.5 Controlador....7
4.6 Service....7
S'encarrega d'interactuar amb la base de dades, xifrar contrasenyes, generar tokens, etc....7
4.7 Routes "routes/usuariRoutes.js"....8
4.8 Executar consultes Postman....8
4.9 Comprovar MongoDB....8
Referències....9
Apèndixs....9
A. Comandes útils de Node/Nodemon....9
B. Fitxer .gitignore recomanat....9
C. Validacions habituals en Mongoose....9
D. Errors comuns i depuració....10
Lliurable de la Sessió 8....10

1. Dates

- 19/11 - 26/11

2. Objectius

Pes	Preferent?	Funcionalitat
0,4	✓	Access Tokens, Refresh Tokens
0,4	✓	Rotación segura y protección contra ataques de reutilización
0,2	✓	Postman/MongoDB

3. Feina prèvia abans de la sessió

Avantatges de l'estratègia avançada d'autenticació amb JWT (Access + Refresh + Rotació)

---

1. Seguretat millorada

- **Tokens signats digitalment:** Permeten verificar que no han estat modificats.
- **Contingut xifrat (opcional):** Protegeix dades sensibles en el payload.
- **Contrasenyes cifrades:** Ús de bcrypt per guardar contrasenyes de manera segura.

---

2. Sessions sense estat (stateless)

- **No s'obre estat al servidor:** La informació està al token.

- **Escalabilitat:** Facilita distribuir la teva aplicació en múltiples servidors sense guardar sessions.

---

3. Rotació segura del refresh token

- **Cada ús genera un token nou:** Cada vegada que es renova, el refresh token es reemplaça.
- **El token antic es fa invàlid:** Reutilització impossible, evitem el reuso maliciós.
- **Reducció de riscos:** Si un token és robat, només es pot utilitzar una vegada.

---

4. Protecció davant d'atacs de reutilització (replay attacks)

- **Tokens d'un sol ús:** S'elimina o invalida un cop usat.
- **Control de la reutilització:** Impossible usar un token robat diferents vegades.

---

5. Control i revocació

- **Llista de refresh tokens en base de dades:** Permet gestionar i invalidar tokens en qualsevol moment.
- **Resposta ràpida:** Tancament de sessions, revocació en cas d'incidents.

---

6. Millor experiència d'usuari

- **Sessió prolongada i segura:** Sense que hagi d'iniciar sessió cada vegada.
- **Renovació automàtica:** Sense posar en perill la seguretat, gràcies a la rotació i protecció.

---

*La combinació de JWT amb rotació segura, eliminació de tokens usats i protecció contra reutilització proporciona un equilibri perfecte entre **seguretat, escalabilitat** i **comoditat** per a l'usuari.*

4. Feina durant la sessió i abans de la següent sessió

4.1 Autenticaciò amb JWT + Token Refresh y Rotació Segura

Objectius:

- Implementar login amb generació de **access token i refresh token**.
- Mantenir la sessió activa amb rotació de **refresh tokens**.
- **Protegir rutes amb middleware** que validi els tokens.
- Millorar la seguretat per a evitar atacs de reutilització (token theft).

Com implementar un sistema de login amb tokens JWT, mantenir la sessió activa amb rotació segura i protegir les rutes

1. Generar un access token i un refresh token en iniciar sessió

Què és?

- Quan un usuari inicia sessió, el servidor crea **dos tipus de tokens**:
    - **Access token:** com una credencial curta que permet accedir a les rutes protegides. Té una durada curta (per exemple, 15 minuts).
    - **Refresh token:** una credencial més gran que serveix per renovar l'accés token quan caduca. Té una durada més llarga (per exemple, 7 dies).

Per què?

- Per reduir el risc: si algú roba l'access token, només podrà usar-lo uns minuts.
- Per mantenir la sessió activa sense que l'usuari hagi de tornar a iniciar sessió constantment.

---

2. Mantenir la sessió activa amb rotació segura de tokens de refresc

Què és la rotació segura?

- Cada vegada que el client demana un nou access token usant el refresh token, **el servidor en crea un de nou** i elimina el precedent.
- Això significa que:
    - Si algú roba un refresh token antic, només podrà usar-lo una vegada.
    - Incrementa la seguretat, perquè els tokens "es renoven" constantment.

Com funciona?

- L'usuari demana renovar la sessió amb un refresh token vàlid.

- El servidor ho verifica, crea un de nou, i elimina l'anterior
- El client rep els nous tokens i continua navegant sense tornar a posar credencials.

---

3. Protegir les rutes amb middleware que valida els tokens

Què és?

- Abans que un usuari pugui accedir a una ruta protegida (com ara el seu perfil o el carret), el servidor **verifica** que el seu token sigui vàlid.
- Aquesta verificació es fa amb un **middleware**: una funció que revisa i valida el token abans d'autoritzar l'accés.

Com funciona?

- El client envia en l'encapçalament (`Authorization: Bearer <token>`) el token d'accés.
- El middleware verifica que:
  - El token sigui correcte i no hagi estat manipulat (firmat).
  - Hagi caducat.
- Si tot va bé, permet que l'usuari segueixi navegant. Si no, denega l'accés.

---

4. Millorar la seguretat contra atacs de reutilització (robo de tokens)

Què és?

- Quan algú roba un refresh token, si només es pot usar una vegada, **no podrà reutilitzar-lo**.
- La **rotació segura** fa que **cada refresh token només funcioni una vegada**, i si algú ho intenta reutilitzar, es s'invalidarà automàticament.

Per què ajuda això?

- Augmenta la **seguretat**, perquè encara que roben un token, només poden fer-lo servir una vegada.
- A més, pots gestionar els tokens a la base de dades i revocar-los manualment si sospites d'activitats sospitoses.

- *Quan algú inicia sessió, rep dues credencials:* `access token` *(curt termini) i* `refresh token` *(més llarg).*
- *El* `access token` *permet accedir a les rutes protegides.*

- *Quan caduca, el client usa el* `refresh token` *per obtenir un de nou, i aquest* ***actualitza el refresh token*** *per un de nou (rotació).*
- *Abans d'accedir a rutes importants, es valida el token amb un* ***middleware****.*
- *La rotació de refresh tokens evita que, si roben un, sigui reutilitzat múltiples vegades, millorant la protecció.*

4.3 Implementar

4.4 Crear l'esquema

…..

……

4.5 Controlador

Només maneja les sol·licituds i delega la lògica a un servei.

…..

4.6 Service

S'encarrega d'interactuar amb la base de dades, xifrar contrasenyes, generar tokens, etc.

…….

4.7 Routes “routes/usuariRoutes.js”

4.8 Executar consultes Postman

4.9 Comprovar MongoDB

Mostra el resultat amb captures de pantalla de les execucions en la teva base de dades en MongoDB

Referències

- What is an ADR? – Michael Nygard
- Awesome ADR – repositori amb exemples
- MongoDB – Documentació oficial
- Draw.io (eina de diagrames gratuïta)

Apèndixs

A. Comandes útils de Node/Nodemon

- `npm run dev` → iniciar en mode desenvolupament.
- `CTRL+C` → aturar servidor.
- Escriure de forma clara i breu, no més d'1 pàgina.

B. Fitxer `.gitignore` recomanat

```
node_modules/

dist/

.env
```

C. Validacions habituals en Mongoose

- `required`
- `minlength`

- `maxlength`
- `enum`
- `unique`
- `match`.

D. Errors comuns i depuració

- No carregar `.env` → MONGO_URI indefinit.
- Port 3000 ocupat → cal canviar `PORT` al `.env`.
- Índexs duplicats si es fan canvis freqüents al model.

Lliurable de la Sessió 8

PDF amb totes les explicacions i link a GitHub.